

Mobile Development :

14 : Revision

10-Slides Revision Cheat sheet



Professor Imed Bouchrika

National School of Artificial Intelligence
imed.bouchrika@ensia.edu.dz

Why Mobile Market is important

- Mobile usage dominates digital activity: most web traffic and searches are from now mobile.
- Users spend the majority of phone time on **apps**, not calls.
- For companies, having a mobile app is no longer optional; it is strategic for visibility, engagement, and competitiveness.
- Major new companies/startups, their main prod or major users are on a mobile app (TikTok, FB, Youtube...)

Key Business/Stat Insights

- Smartphones are used by the vast majority of the global population.
- App stores contain **millions of apps**, but **very few succeed**.
- Performance is critical: Slow loading (>3 seconds) and crashes lead to uninstallations.
- **Android dominates globally**, iOS is strong in premium markets
- Most revenue comes from **free apps**, not paid downloads.
- Mobile apps generate revenue through: Advertising + In-App Purchases (IAP) + Subscriptions

Dart Programming

- Client-optimized, object-oriented language by Google. Main language used by **Flutter**.
- Statically typed with **type inference**.
- Supports: Null safety + Asynchronous programming + Object-Oriented Programming (OOP)
- Type-test operators : is

- **int**, **double**, **String**, **bool**, **num**
- **var**: inferred type (cannot change type later)
- **dynamic**: avoid unless necessary
- **late** : to be assigned a value later.
- **const**: compile-time constant
- Null safety: **String?** allows null
- **List**: ordered, allows duplicates [1, 2, 3,1]
- **Set**: unordered, unique values {1, 2, 3}
- **Map**: key-value pairs {'key': 'value'}

Types of Mobile Apps

- **Native App** : Built specifically for one platform (Android or iOS). Platform-specific languages and SDKs. + Best Performance + Full access to device hardware + Optimized ux + offline access -high cost - maintenance - multiple codebases
- **Web Apps** : Mobile-friendly websites accessed via a browser : +Easy/Fast/Cheap Dev + Cross-Platform + Maintenance: - Limited hardware access - Performance.
- **Hybrid / Progressive Web Apps** : Web technologies packaged as installable apps and runs inside a native container : + one codebase + low cost + offline: -Performance + Complexity with advanced features

- Arrow Function syntax: **int add(int a, int b) => a + b;**
- Anonymous functions (lambdas)
- Named parameters (void greet({String? name}))
- Callbacks: functions passed as params
- **!**. Certain that the object is not null
- **?**. If this object is not null, proceed; otherwise return null.
- **??** Use this value if the first one is null.

Mobile App for a Business

- A successful app = Real problem with impact+ Solution/Value + validated market + good UX + sustainable business model
- Building an app ≠ building a business, but **successful apps require both**.
- App stores (Google / Apple) **remove poorly planned apps**.
- Software Engineering aims to **produce high-quality software** through **well-defined processes**, not ad-hoc coding.

Mobile App Dev Process: 1 Idea & Market Research

- How to get ideas : Identify **real problems** from our daily life / Follow your **passion** / Study **existing apps** (To improve, localize..) / Analyse user reviews + Market trends + keyword search volume
- **Ideas don't need to be original** → innovation often = improvement.
- Find Idea → Market Research (Competition, Market..) → Idea Validation (What, Who, Why Where, How ..?)

2 : Analysis & Planning

- Requirements Engineering : Collect Functional requirements + non-functional requirements (Performance / Scalability / Offline-Online/Security/Storage/Sync..) + End-User Analysis (Skills, language) + Context (Connectivity, env, devices)
- Roadmap/Budget Plan are produced

5. Testing & Quality Assurance

- UX & usability testing
- Unit/Component/Integration Testing
- Security/Performance/scalability test
- Testing is continuous especially / updates

6. Distribution & Maintenance

- Publish on app stores or deploy privately.
- Monitor for reviews/rating/logs
- Fix Bugs and improve continuously.

3. UI & UX Design

- Wireframes/Screens Design, Navigation flows, Prototypes & mockups, Visual identity (colors, fonts, icons) : delivered.
- Mobile apps prioritize:
- **Usability** = Ease of use (Learnability Efficiency, Memorability, Error handling, Satisfaction
- UX = how a person **feels when using an app** (Usability + Adoptability (will users return?) + Desirability (fun, engagement) +Value (does it help?))
- Nielsen's 10 Usability Heuristics (Must Know): Visibility of system status, Match with real world, User control & freedom, Consistency & standards, Error recovery , Error prevention, Recognition over recall, Flexibility & efficiency, Minimalist design, Help & documentation

4. Development

- Architecture design (frontend, backend, APIs, DB)
- Technology stack selection
- Implementation & integration

Business Models for Mobile Apps

- A business model = **how you capture value**, not cost.
- No single best model → depends on app & users.
- Common models: Up-front payment / Subscription / Usage-based (pay-as-you-go) / Freemium / Advertising / Upselling / Franchise / White-label / “Parking meter” (low base, high penalties)

Coding Principles for Mobile Apps

- Code must be **readable and maintainable**, not just correct.
- Reality: **1 minute writing code** → **~9 minutes reading it**.
- Naming of variables, classes, functions : Intention-revealing names + Pronounceable + searchable. : Variables → camelCase, Class → PascalCase, Function → camelCase + Starts with a verb (get.. , set, check, compute..)
- Proper indentation is mandatory + Avoid Files with many lines (avoid horizontal/vertical scrolling)
- Organize files by: **Domain** (sales, users, auth...) or **Technical role** (screens, widgets, services)
- Code smaller blocks → Compile frequently → debug early
- Key Principles : SOLID, KISS, DRY

Introduction to Flutter

- Open-source UI framework by **Google**, Uses **Dart**
- Single codebase → Android, iOS, Web, Desktop, Compiles to **native code** (high performance) + Uses **Skia rendering engine** for consistent UI [Dart code is **compiled to native BUT Flutter does NOT use native UI widgets**, uses Skia]
- + Google & community support + Fast Dev + Performance + Rich Lib
- **void main() { runApp(const MyApp()); }**
- Everything is a Widget : App, Screen, Button, text, spacing = widgets
- **Code Widgets** : **MaterialApp** [App-wide configuration (theme, routes, navigation)] + **Scaffold** [Basic screen structure (AppBar, body, FAB, drawer)]
- **Flutter = Widget Tree + State Management + Clean Structure**

Exceptions/Debug

- Use **try-catch** to handle runtime errors safely- Print stack trace
- You can use print to debug but make sure you remove them or encode them for debug printing mode only.
- Recommended to use **logger** package for structured logs (**debug**, **info**, **warning**, **error**)

Stateless Widget

- Immutable, Built once, Used for static UI (text, icons)

Stateful Widget

- Mutable, Has a separate **State** class, UI updates using **setState()**, Used for dynamic UI components
- Access widget properties inside state : **widget.propName**
- Cycle : createState() → initState() → didChangeDependencies() → Build() → SetState → didUpdateWidget() → deactivate() → dispose()

Dev using Flutter

- **Keep main.dart minimal** + Separate screens & widgets → files
- Pubspecc.yaml Manages, Dependencies (Release + dev) + Assets (images, fonts) [^ caret means **backward compatible**]
- flutter run
- flutter clean
- flutter pub add package

- Layout : Row / Column / Expanded / Spacer / SizedBox [fixed spacing] / Stack + Positioned [overlay & absolute positioning]
- Containers: Container, Card, Padding, Center, Align
- UI : Text / Card / ListTile / ..
- Input: TextField, TextFormField, ElevatedButton, IconButton, GestureDetector
- Lists & Data : ListView.builder Scrollable + Lazy loading
- Style: TextStyle, BoxDecoration.

How Interactivity Works in Flutter

- With a stateful widget, define the state variables (Choose the data structure that better describes the context/app)
- Update them based on user events/Timer
- Call `setState({})` so that the Flutter builds the Widget based on the updated variables data.
- **initState()** Called once Used for initialization
- **build():** Called many times, Used only for UI rendering
- Never put logic or initialization inside **build()**.
- **Case Studies : Tic-Tac-Toe, Water Sorting Puzzle, Counter**

Navigation Between Screens : Direct

- No named addresses, just use :
`Navigator.push( context, MaterialPageRoute( builder: (_) => Screen(data: value),,);`

Navigation Between Screens : Dynamic

- Use property **`onGenerateRoute: (settings) {}`** of `MaterialApp`
- Do the business logic based on the settings variable to route to the correct screen:
**`onGenerateRoute: (settings) {
 switch (settings.name) {
 case 'profile':
 if (!isLoggedIn) {return
MaterialPageRoute(builder: (_) => LoginScreen());
}
}`**

Navigation Between Screens : Named Routes

- For navigation between screen, Flutter uses a Stack (Push / Pop)
- Named Routing is cleaner, scalable & maintainable for complex app
- Each screen must has an address as a String:
`static const routeName = '/screenName';`
- The routes properties within the material app should be defined to include all named screens
**`routes: {
 WelcoScreen.routeName: (context) => const WelcoScreen(),
 DoubleScrn.routeName: (context) => DoubleScrn(number: 0)},`**
- To take a user to another named screen:
`Navigator.pushNamed(context, ScreenClassName..routeName);`
- To go to the previous screen : **`Navigator.pop(context);`**
- Use **`pushAndRemoveUntil`** to clear the entire nav stack and take the user to another new named screen
- To go the first screen :
`Navigator.popUntil(context, (route) => route.isFirst);`

Passing Data Between Screens

- Parent → Child : 1) Pass data via **constructor** 2) via arguments of the `pushNamed`:
`Navigator.pushNamed( context, Screen.routeName, arguments: value);`
To get data: **`ModalRoute.of(context)!.settings.arguments`**
- Child → Parent : In child : **`Navigator.pop(context, result);`**
Parent : final result = `await Navigator.push(...);`

Form Validation in Flutter

- A key for the form must be defined:
`final _formKey = GlobalKey<FormState>();`
- Enclose input elements inside Form widget + assign key: `_formKey`
- Use the validator attribute within the Input widget:
**`TextFormField(`
 `validator: (value) {`
 `if (value == null || value.isEmpty) return 'Error msg';`
 `return null; }` //null for valid**
- Upon submitting the form, involve the validate method of form:
`if (_formKey.currentState!.validate()) { /* process */ }`

FutureBuilder

- is a Flutter widget that builds UI based on the state of a Future object. It automatically updates the UI when the Future: is loading and completes successfully. ONE TIME ONLY.

```
FutureBuilder(  
  future: future_object,  
  builder: (context, snapshot) {  
    if (snapshot.connectionState ==  
ConnectionState.waiting) {  
      return CircularProgressIndicator(); }  
    if (snapshot.hasData && snapshot.data != null) {  
      return Text('some data or listBuilder..');  
    } else if (snapshot.hasError) {  
      return Center(child:  
Text('Error:${snapshot.error}'));;}  
    return const Center(child: Text('No Data Avail'));
```

Async Programming

- Asynchronous programming allows **long-running operations** (such as network calls, file I/O, database access, or heavy computations) to run **without blocking the main UI thread** [*is executed on separate event loop, not separate thread*]
 - **async**: Declares function as asynchronous.
 - **await**: Makes async call synchronous (waits for completion). **Must be used inside** async functions.
 - **Future<T>**: Return type for async functions (String → Future<String>, bool → Future<bool>..)
- ```
Future<bool> getData() async {
 await someFunc();
 return true;
}
```

- To call an async function : :  
**`String myData = await getData();`** // Called as a **sync function**  
**`Future<String> myData = getData();`**

## Pull to Refresh Widget

```
RefreshIndicator(
 onRefresh: _refreshData,
 child: ListView(...),
)
Future<bool> _refreshData() async {
 myInfo = loadData();
 setState(() {});
 return True;
}
```

## Time Functions

- Periodic Timer  
**`Timer.periodic(Duration(seconds: 30), (Timer t) {`**  
    **`_refreshData();`**  
    **`});`**
- Blocking Wait time.  
**`await Future.delayed(Duration(seconds:5));`**

## SharedPreferences

- Used to store data locally on the phone in the format key:value. The key must be a string whilst the value can be any primitive data type(int, string, bool) + List<Str>.
- **Persistent** ; data remains after app is closed.
- Used to store **simple data** such as: Login state (isLoggedIn), App settings (theme, language), Counters, flags, small user preferences
- Do **NOT** use it for Large data/Complex objects.
- 1) Initialize in initState() and call setState() after loading to update UI. 2) Separate always the UI from Business Logic. 3) For keys, use constants to avoid typos.

## Starting with SharedPreferences

- Add dependency **shared\_preferences**  
It's available on all plat(Android,ios,win,linux,web)
- Create/Access instance of the sharedpref library:  
**final prefs = await SharedPreferences.getInstance()**
- To write data setInt, setString, setBool(key, value)  
**await prefs.setInt('counter', 10); // async func.**
- To get data from sharedprefs: getInt, getString..  
**final counter = prefs.getInt('counter') ?? 0;**  
It is not an async function
- To remove data : **await prefs.remove('counter');**
- To clear all data : **await prefs.clear();**

## SQLite Database (sqlite) : Setup/Helper

- Used to store relational data with tables where each data element stored as a row within a table.
- To get started, you need to install the dependencies: flutter pub add sqflite path For Windows/Linux: add flutter pub add sqflite\_common\_ffi
- Create the DBHelper which has functions to : create the database tables on the first install ( onCreate) in addition to executing specific business logic when upgrading the database version.

```
class DBHelper {
 static const _db_name = "MY_DB.db";
 static const _db_version = 1;
 static var database;
 static Future getDatabase() async {
 if (database != null) return database;
 database = openDatabase(
 join(await getDatabasesPath(), _db_name),
 onCreate: (db, version) {
 db.execute("
 CREATE TABLE todo (
 id INTEGER PRIMARY KEY AUTOINCREMENT,
 title TEXT
)");
 },
 version: _db_version,
 onUpgrade: (db, oldV, newV) { ... },
);
 return database;
 }
}
```

## SQLite CRUD

```
final db = await DBHelper.getDatabase();

final data = await db.rawQuery(
 'SELECT * FROM todo');

await db.insert('todo',
 {'title': 'Task', 'done': 0},
 conflictAlgorithm: ConflictAlgorithm.replace);

await db.rawQuery('UPDATE todo SET
done=? WHERE id=?', [1, 5]);

await db.rawQuery('DELETE FROM
todo WHERE id=?', [5]);
```



## Why State Management?

- State management is the process of updating the UI automatically when data changes, even across: Multiple widgets or Multiple screens
- Using `setState()` is simple but not scalable + Hard to maintain + cannot easily share state across widgets/screens.

## What is BLoC/Cubit

- Cubit/BLoC(Business Logic Component) is a Flutter state-management pattern that separates business logic from UI, using emitted states to automatically rebuild widgets in a predictable and scalable way
- Cubit = simplified BLoC (Without events)
- To use it : `flutter pub add flutter_bloc`

## Cubit States as Class

- Maps are used at the cubit state or small prototypes/quick demos/temp coding.
- Instead of using Maps as a State for the Cubit, a custom class can be created instead as : Type safety at compile time + better readability/maintainability.(Classes are self-documenting/IDE vs. Maps
- Classes works better with `Equatable` to avoid unnecessary + compares states efficiently

## Cubit/BloC Step

- Create the Cubit Component  

```
class CounterCubit extends Cubit<int> {
 CounterCubit() : super(0);
 void increment() => emit(state + 1);
 void decrement() => emit(state - 1);
}
```
- Use the Provider for provision an instance of Cubit within the widget tree.
- To invoke a method for the Cubit, locate it:  

```
ElevatedButton(
 onPressed: () {
 context.read<CounterCubit>().increment();
 },
 child: Text('Increment'),
)
```
- Cubit executes Business Logic ( + call backend, db..) and emits a new state.
- The `BlocBuilder`, `BlocConsumer` will be refreshing the UI when receiving a different state.

```
abstract class LoginState extends Equatable{
 @override
 List<Object> get props => [];
}

class LoginInitial extends LoginState {
 @override
 List<Object> get props => ['loading'];
}

class LoginSuccess extends LoginState {
 final String username;
 @override
 List<Object> get props => ['logged'];
 LoginSuccess(this.username);
}
```

## Cubit/BloC UI Widgets

- **BlocProvider** or **MultiBlocProvider** :  
Makes Cubit available to the widget tree .  

```
MultiBlocProvider(
 providers: [
 BlocProvider(create: (_) => CounterCubit()),
 BlocProvider(create: (_) => ThemeCubit()),
],
 child: MyApp(),
)
```
- `BlocBuilder` ; Rebuilds UI on every different state emitted : Like `Obx` in `GetX`  

```
BlocBuilder<CounterCubit, int>(
 builder: (context, count) {
 return Text('$count');
 },
)
```
- `BlocListener` : Reacts to state changes (not build UI). Used for side-effects business logic (Snackbar, navigation..)
- `BlocConsumer` : Combines `BlocBuilder` + `BlocListener`  

```
BlocConsumer<Cubit, State>(
 listener: (context, state) {},
 builder: (context, state) {},
)
```

```
class LoginCubit extends
Cubit<LoginState> {
 LoginCubit() :
 super(LoginInitial());
}
```



## Background Jobs

- Background jobs are pieces of code that run **outside the main UI thread**, ensuring that the application remains **responsive**.
- It Can run: When app is closed + Periodically + At scheduled time
- Scenarios : Keep UI responsive while executing intensive tasks + Sync data with remote servers + Upload files + Collect Analytics + Audio playback + Checking updates/notifications.
- Libraries:
  - **workmanager** : Android/iOS, runs when App closed + Min recurring is 15 mins
  - **android\_alarm\_manager\_plus** : Android only, Runs when App is closed, Exact time
  - **cron** : runs in all platforms but only when the app is running, Exact Timing.

## Architecture : Direct Access

- Mobile Apps can connect directly to cloud/external services without the need for the custom/dedicated backend
- **Pros** : Fast development Simple setup
- **Cons** : Credentials embedded in app → security risk + Business logic forced into mobile app + Hard to change DB/service provider/schema + Vulnerable to reverse engine
- Used Mainly for simple and disposable apps + no long term maintenance.

## Architecture : Via Backend Access

- Mobile app talks to API endpoints hosted on dedicated backend which access the external services/databases.
- The Backend handles: Data validation & curation + Security + Business logic (ML, rules, workflows) + Integration with other services
- The backend acts as a **middle layer** between app and external services/db.
- **Pros** : Secure Flexible Easy provider change Scalable
- **Cons** : More complexity Longer development time

## Repository Pattern : Where data comes from

- Separates data access logic from the rest of the app.
- How it works: The UI or state layer talks to a Repository + The repository decides where data comes from (API, database, cache)
- Benefits: Cleaner architecture + Easier testing (can mock data sources) + Easy to switch data source (e.g. Firebase → REST API)

## Service Locator Pattern : How services are obtained

- Provides a **central registry** to access app services and dependencies
- How it works: Services are registered once + Requested from anywhere when needed + Commonly implemented using **GetIt**
- Benefits: Avoids passing objects through constructors + Centralized dependency management

## Singleton Pattern : Ensure one shared instance

- Ensures that **only one instance** of a class exists during app lifetime.
- Benefits: Saves memory + Maintains shared state (e.g. settings, DB connection) + Consistent access point

## Introduction to Testing

- Testing is the process of: Verifying the software does what it should + Finding defects early in the life cycle + Establish confidence software fit to use.
- **Testing never proves absence of bugs** "Testing can show the presence of errors, not their absence."
- Testing is Critical ? Reduces development & maintenance cost/ Improves user satisfaction & trust/ brand reputation / Prevents production disaster
- **Testing**: Finding defects (anyone can do it)
- **Debugging**: Locating & fixing defects (developer only, costly + very difficult)
- **Regression Testing**: Re-run tests after: Bug fixes, New feature, Conf chang
- **TDD**: Write tests before code → code becomes more testable.
- Testing must be done on **Staging Env** not on production or dev environments

## Unit Testing

- Tests **smallest units** in isolation / For Functions, classes, business logic/ Isolation: No UI, no network, no db (Use **mocktail** to isolate)
- Important : Early bug detection / Fast & cheap / Safe refactoring
- DFO Deterministic, Fast, Offline

```
blocTest<CounterCubit,
Map<String, dynamic>>('Test
Cubit',
build: () => CounterCubit(),
act: (cubit) =>
cubit.increment(),
expect: () => [
 (state, loading, counter)
```

```
test('Counter Test 1', () async {
 WidgetsFlutterBinding.ensureIniti
alized();
 SharedPreferences.setMockInitialV
alues({});
 final counter = Counter();
 await counter.init();
 await counter.increment();
 expect(counter.getValue(), 1);});
```

## Widget Testing

- Tests UI widgets in isolation
- Verifies: Layout+ User interactions + State changes
- Runs in a simulated env
- Tip! Use Key to locate widgets

## Main Testing Classifications

- **Static**: Code/design review without execution **vs** **Dynamic**: Testing while the app is running
- **Passive**: Analyze logs & traces (production monitoring) **vs** **Active**: Interact directly with the app
- **Positive**: Valid inputs, expected paths **vs.** **Negative**: Invalid inputs, edge cases (stability testing)
- **Black Box**: No code knowledge (functional testing) **vs.** **White Box**: Code-aware testing (logic & paths)
- **Functional**: Features work? (login, signup, CRUD) **vs.** **Non-Functional**: Quality attributes: Performance, Security, Usability, Reliability, Accessibility,
- **Manual Testing** Human-driven + Repetitive & error-prone **vs.** **Automated Testing** : Uses test scripts & frameworks. Faster, repeatable, reliable Essential for CI/CD , Enables Test-Driven Development (TDD)

```
testWidgets('Widget Test', (WidgetTester tester)
async {
 await tester.pumpWidget(const MyApp());
 await tester.enterText(find.byKey(Key('tx_input1')), '3');
 await tester.tap(find.byKey(Key('bt_result')));
 await tester.pumpAndSettle(); ...
 expect(resultWidget.data, '6');
```

## Integration Testing / E2E

- Tests Tests complete user flows / Multiple widgets + logic + navigation / Closest to real user behavior

## App Config Basics

- App Identifier: Unique string (e.g., dz.edu.ensia.mycounter) - CANNOT change once published or treated as new app + Stored inside android/app/build.gradle
- Version Format: 1.2.3+100 (major.minor.patch+buildNumber)
  - Major: Breaking changes / Minor: New features / Patch: Bug fixes only
  - Stored inside the pubsec.yaml
- Displayed App Name The application has the display name that is shown to the user below the icon : Store inside android/app/src/main/AndroidManifest.xml under android:label
- Localizable App Name Per language using strings.xml in values/values-AA

## Security Measures

- flutter build appbundle --release --obfuscate --split-debug-info=./debug-info**
- Obfuscation: Renames code to meaningless names (prevents reverse eng)
  - Minification: Removes whitespace/comments (20-40% size reduction)
  - Debug Symbols: Saved separately to debug crashes while keeping app small

## Signing Apps

- App signing is the process of digitally signing an app to Verify the developer identity → Ensures updates come from the **same developer**
- **Keystore** : is a file containing private signing keys (Key alias). There are passwords to protect Keystore+Keys
- If signing key is lost → App cannot be updated.

## App Distribution

- Direct Distribution ( outside stores) : Host APK on website (users enable "Unknown Sources") + Dev must device update mechanism + Low visibility
- App Stores : Google Play Store: \$25 one-time fee, 2.5B+ devices, auto-updates + Larger audience /Apple App Store: \$99/year, stricter review : Premium users

## Compile Types

- JIT – Just-In-Time Compilation : Hot reload + Fast Dev - slow runtime - larger memory usage. Used for debug
- AOF - head-Of-Time Compilation : Code is compiled **before execution** into native machine code. Optimized native binaries + Faster : Used for release

## Build Modes

- Debug build: Development & testing + Hot Reload + Debugging tools + Detailed logs + Slow performance + Not publishable + Used while writing/ testing code.
  - **flutter build apk --debug / flutter run**
- Profile : Purpose: Performance testing/Debugging disabled+Performance tracing enabled+ Near release performance + No Hot Reload + Not publishable + Used to analyze app performance (CPU, memory,...)
  - **flutter build apk --profile / flutter run --profile**
- Release build: Production & publishing / Fully optimized + Debug code & logs removed + Smallest size + Best performance + Required for app stores
  - **flutter build apk --release / flutter run --release**
  - **flutter build appbundle**

## Build Formats

- APK: Traditional Android package + Larger size
- AAB (Android App Bundle): Recommended by Google → Google generates optimized APKs per device + Smaller download size + Google Play requires AAB